Introduction to Git

1 What is Git?

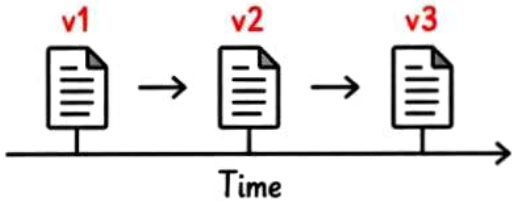
Git is a **DISTRIBUTED VERSION CONTROL SYSTEM** (DVCS) used to track changes in source code during software development.

Why Git?

- Track every change
- Work with team easily
- Go back to any version
- Create branches & features
- Safe, fast & reliable
- Open Source & Free

2 What is Version Control?

Version Control is a system that records changes to files over time so you can recall specific versions later.



3 Git vs GitHub

Git	GitHub
• Tool (Version Control System)	• Platform (Cloud Service)
• Works on your local machine	• Hosts Git repositories
• Command line based	VS. Web based interface
• Manages code history	• Collaboration & sharing

4 Benefits of Git

- **Speed**: Fast operations & performance.
- **Safety**: Full history of changes, easy to recover.
- **Collaboration**: Multiple developers can work together smoothly.
- **Efficiency**: Branching, merging & rollback make development easier.
- **Open Source**: Free, powerful & used worldwide.

5 Where is Git Used?

- **Software Development**
  Track code changes & versions.
- **DevOps & CI/CD**
  Manage infrastructure, automation & deployment scripts.
- **Documentation**
  Track changes in docs, blogs, wikis.
- **Game Development**
  Track assets, code & game versions.

Remember
Git tracks changes, not just files.
It's like a time machine for your code!

Tip
Commit small, commit often.
Write meaningful commit messages.

Git Architecture

Git works like a 4-layer system where your changes move step by step from your computer to the remote server.

① Working Directory

Where you edit and work on your files.
(Untracked/Modified)

git add →

② Staging Area

Changes added and ready to be committed.
(Tracked)

git commit →

③ Local Repository

Changes are committed and saved locally.
(.git directory)

git push →

④ Remote Repository

Remote server like GitHub, GitLab, Bitbucket etc.
(Shared with team)

Git Lifecycle

Modified

You modify a file.

→

Staged

You add changes to staging area.

→

Committed

You commit changes to local repository.

→

Pushed

You push changes to remote repository.

Easy Analogy

- Working Directory → Your Desk
- Staging Area → Your Clipboard
- Local Repository → Your Locker
- Remote Repository → Your Cloud Locker (Team Storage)

Common Actions in Flow

1. `git status` → Check current state
2. `git add <file>` → Move to staging area
3. `git commit -m "message"` → Save locally
4. `git push origin <branch>` → Push to remote

Remember

Git helps you track changes, collaborate, and manage code like a pro developer!

Installation & Configuration

1 Install Git

Download and install Git from the official website.

Official Site: https://git-scm.com/downloads

Windows macOS Linux

Follow the installation wizard for your operating system.

2 Check Git Installation

Verify Git is installed successfully.

```
$ git --version
git version 2.44.0.windows.1
```

Tip:
If command is not found, restart your terminal.

3 Configure Username

Set your global username.

```
$ git config --global user.name "Your Name"
  # sets the username
```

This name will be attached to all your commits.

4 Configure Email

Set your global email.

```
$ git config --global user.email "youremail@example.com"
  # sets the email
```

This email will be used for all your commits.

5 View Current Configuration

Check your current Git configuration.

```
$ git config --list
# shows all global
  configuration settings
```

6 Set Default Branch

Set default branch (new repositories).

```
$ git config --global init.defaultBranch main
  # sets the default branch name
  # new repositories will start with 'main'
```

main

Best Practice

- Set your name & email correctly.
- Use a professional email.
- Use meaningful name.
- Check configuration before starting any project.

Global vs Local Configuration

Global (All Repositories)	VS	Local (Single Repository)
Applies to all repositories on your machine.		Applies to only that specific repository.

Remember Good configuration makes your commits consistent and your collaboration smooth!

Essential Git Commands

These are the most important Git commands every developer should know.

1 git init

Initialize a new Git repository.

```
$ git init
# Creates a new .git folder
```

2 git clone <url>

Clone an existing remote repository.

```
$ git clone https://github.com/user/repo.git
# Copies the entire repository
  to your local machine
```

3 git status

Check the status of your working directory.

```
$ git status
# Shows modified, staged
  and untracked files
```

4 git add <file>

Stage changes from working directory to staging area.

```
$ git add index.html
# Stages a specific file
$ git add .
# Stages all changes
```

5 git commit -m "message"

Commit staged changes to local repository.

```
$ git commit -m "Initial commit"
# Saves changes with a
  meaningful message
```

6 git log

Show commit history.

```
$ git log
# Shows all commits
$ git log --oneline --graph
# Compact & beautiful view
```

7 git diff

Show differences between changes.

```
$ git diff
# Working directory vs
  staging area
$ git diff --staged
# Staging area vs last commit
```

8 git branch

List, create or delete branches.

```
$ git branch
# List all branches
$ git branch feature/login
# Create new branch
```

9 git checkout <branch>

Switch to another branch.

```
$ git checkout main
# Switch to main branch
```

Quick Tips

- Use git status often.
- Add changes (git add) before commit.
- Write meaningful commit messages.
- Commit small, commit often.

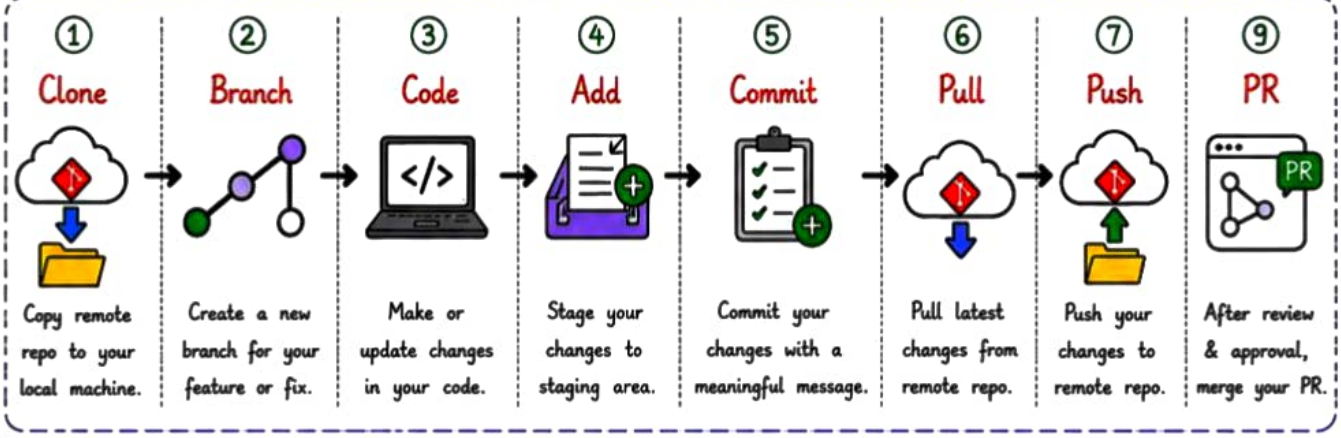
Remember

Add → Commit → Push

This is the heart of Git workflow!

Complete Git Workflow

Here is the complete end-to-end Git workflow you follow in any real project.



A More Detailed Flow

①	`git clone <url>`	# Clone remote repository
②	`git checkout -b <branch>`	# Create branch
③	Make changes in file	# Edit, add or update
④	`git add <file>`	# Stage changes
⑤	`git commit -m "message"`	# Commit changes
⑥	`git pull origin <branch>`	# Get latest changes
⑦	`git push origin <branch>`	# Push changes
⑧	Create Pull Request (PR)	# Review changes
⑨	Merge PR to main branch	# Complete workflow

Example Scenario

`git clone repo.git`	Clone the repository
`git checkout -b feature/login`	Create feature branch
`# Edit code`	Make your changes
`git add .`	Stage all changes
`git commit -m "Add login page"`	Commit changes
`git push origin feature/login`	Push branch to remote
Create Pull Request	Ask for review
Merge to main	After approval, merge

Best Practices

- Always work on a separate branch.
- Pull latest changes before you start coding.
- Commit small, meaningful changes.
- Write clear and descriptive commit messages.
- Review your code before creating PR.
- Keep your main branch clean and protected.

Tips

- Sync your branch with main regularly.
- Resolve conflicts locally before pushing.
- Use descriptive branch names.
- Close PR after merge.

Remember: A clean workflow = Better Collaboration + High Quality Code

Git Branching

A branch in Git is just a lightweight movable pointer to a specific commit.

1 What is Branch?

A branch allows you to work on new features, fixes or experiments without affecting the main code.

2 Why Use Branches?

- Work on multiple features in parallel
- Keep main branch clean and stable
- Easy to test new changes
- Safe collaboration with team

3 Common Branch Types

Main / Master
Production ready code lives here.

Feature
New feature development.

Release
Prepare for a new release.

Hotfix
Fix critical issues in production.

Develop
Integration branch for development.

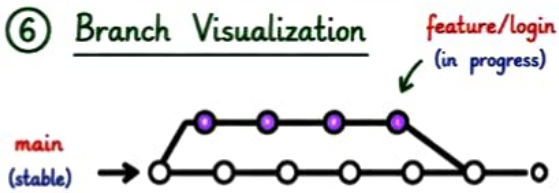
4 Basic Branching Commands

`git branch`	→ List all local branches
`git branch <name>`	→ Create a new branch
`git branch -d <name>`	→ Delete a branch
`git checkout <name>`	→ Switch to a branch
`git switch <name>`	→ Switch to a branch (new)
`git checkout -b <name>`	→ Create & switch to a new branch

5 Branching Workflow

Create a new branch →	`git checkout -b feature/login`
Work on your feature →	Edit, Add, Commit
Push to remote →	`git push origin feature/login`
Create Pull Request (PR) →	Open PR on GitHub
Merge to main →	Merge PR to main branch

6 Branch Visualization



Work happens on feature branch and then merged back into main.

Tips

- Always create a new branch for new work.
- Keep branches small and focused.
- Delete branches after they are merged.

Remember

Branches help you experiment, build, and collaborate without breaking the main code.

Best Practices

- Use meaningful branch names.
- Pull latest changes before starting work.
- Don't work directly on main branch.
- Review code before merging.

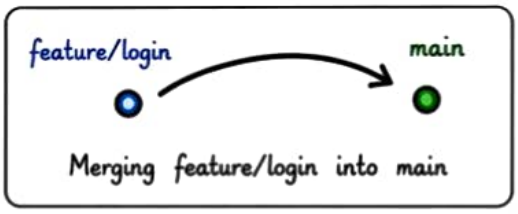
Remember: A clean workflow = Better Collaboration + High Quality Code

Git Merge

Merge is the process of combining changes from one branch into another.

1 What is Merge?

Merge integrates the changes of one branch into the current branch.



2 Types of Merge

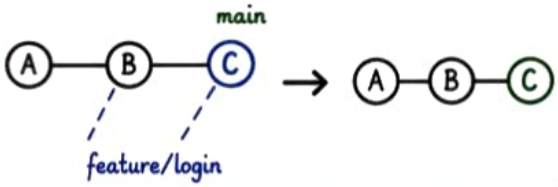
- Fast Forward Merge – No new commit created.
- Three Way Merge – New merge commit created.
- Merge Commit – Keeps history of both branches.

Tip:
Prefer Fast Forward Merge when possible to keep history clean.

3 Fast Forward Merge

When there are no new commits in the main branch, Git simply moves the pointer forward.

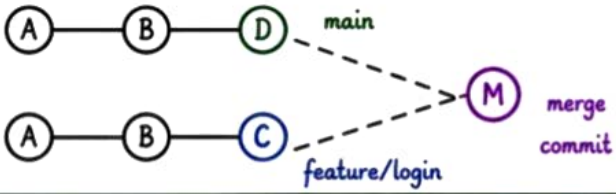
```
git checkout main
git merge feature/login
```



4 Three Way Merge

When both branches have different commits, Git creates a new merge commit.

```
git checkout main
git merge feature/login
```



5 Merge Commit

Creates a new commit to record the merge. Useful for tracking the history.

```
git checkout main
git merge feature/login
```

6 Merge Workflow

Create & switch to feature branch →	`git checkout -b feature/login`
Make changes & commit →	`git add .` `git commit -m "message"`
Push branch →	`git push origin feature/login`
Switch to main branch →	`git checkout main`
Merge branch →	`git merge feature/login`

Advantages

- Keeps code history organized.
- Ensures all changes are tracked.
- Safe and easy collaboration.
- Works well in team projects.

Remember

- Always pull latest changes before merging.
- Resolve conflicts before committing.
- Delete merged branches to keep repo clean.

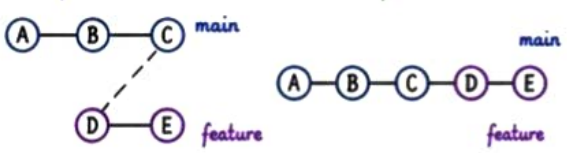
Remember: A clean workflow = Better Collaboration + High Quality Code

Git Rebase

Rebase is the process of moving or combining a sequence of commits to a new base commit.

1 What is Rebase?

Rebase rewrites history by moving the current branch to a new base commit.



After rebase, commits look like they were made on top of the latest base.

2 Rebase vs Merge

Merge	Rebase
• Creates a merge commit	• Rewrites history
• History is not clean	• History is clean & linear
• Easy and safe	• Can be risky
• Good for public branches	• Good for private branches

VS

Tip:
Use rebase for clean history.
Use merge to avoid rebasing public branches.

3 Basic Rebase Command

```
$ git rebase <branch>
# Moves current branch commits
# to the latest commit of <branch>
```

Example:

```
$ git rebase main
# Rebase current
branch on top of
main
```

4 Interactive Rebase

Powerful way to edit, reorder, squash commits.

```
$ git rebase -i <commit>
# Edit commit, drop, reorder, squash
# Reorder, squash, remove, reword
```

5 Common Rebase Use Cases

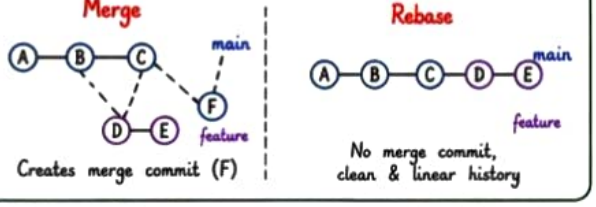
- Keep history clean and linear
- Squash multiple commits into one
- Fix commit messages
- Remove unwanted commits
- Move feature branch on latest changes

Remember:
Rebase changes commit ID.
Use carefully and never on shared branches!

6 Rebase Workflow

Checkout feature branch →	git checkout <feature/login>
Fetch latest from remote →	git fetch origin
Rebase on latest main →	git rebase origin/main
Resolve conflicts (if any) →	Fix Resolve & continue
Push with force (if needed) →	git push --force-with-lease

7 Rebase vs Merge (Visual)



Advantages

- Clean & linear history
- Easier to read
- Looks professional
- Better for review process

Disadvantages

- ✗ Rewrites history
- ✗ Can cause issues if used on public branch
- ✗ Conflicts may occur

Best Practice: Rebase your feature branch on latest main before creating Pull Request.
This keeps your PR clean and easy to review.

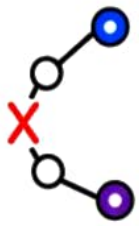
Remember: A clean workflow = Better Collaboration + High Quality Code

Git Conflicts

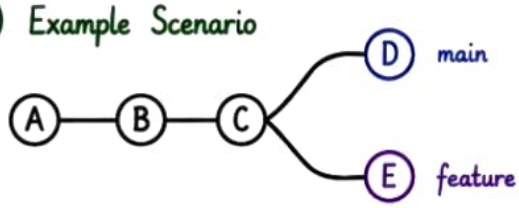
A conflict occurs when Git is unable to automatically merge changes in the same part of the same file.

1 When Do Conflicts Happen?

- When the same file is modified in two different branches.
- When Git doesn't know which change to keep.



2 Example Scenario



If both D and E modify the same line in the same file, a conflict occurs when merging.

3 How Conflicts Look in a File

```
<<<<<<< HEAD (main)
Line changed in main branch
=======
Line changed in feature branch
>>>>>>> feature
```

→ Change from current branch

→ Change from other branch

4 Steps to Resolve Conflicts

1. `git status` → See which files have conflicts
2. Open the file → Look for conflict markers
3. Edit the file → Keep the correct changes and remove markers
4. `git add <file>` → Mark conflict as resolved
5. `git commit` → Commit the merge

5 Conflict Resolution Example

Before (with conflict)

```
<<<<<<< HEAD (main)
Hello from main branch
======
Hello from feature branch
>>>>>>> feature
```

→

After (resolved)

Hello from main branch
+
Hello from feature branch
(combined)

Tip: You can keep one change, both changes, or modify as needed – just make sure to remove the conflict markers.

6 Useful Commands for Conflicts

Command	Purpose
`git status`	Show files with conflicts
`git diff`	See changes in conflicted files
`git add <file>`	Mark file as resolved
`git commit`	Complete the merge after resolving
`git merge --abort`	Abort merge and return to previous state

Best Practices

- Pull latest changes before starting work.
- Communicate with your team.
- Resolve conflicts as soon as possible.
- Test after resolving conflicts.
- Understand the code before deciding.

Remember

- Conflicts are normal in team work.
- Stay calm and read the changes carefully.
- Understand the code before deciding.
- Clean commits = Happy team!

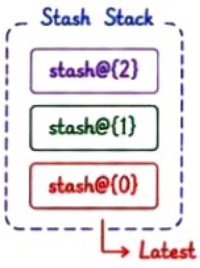
Key Point: Conflicts mean Git needs your help to decide what stays. You are in control!

Git Stash

Git Stash is used to save your uncommitted changes temporarily so you can work on something else.

1 What is Git Stash?

Stash takes your uncommitted changes (tracked or untracked files) and stores them in a stack.



2 When to Use Git Stash?

- You are working on a feature but need to switch to another branch urgently.
- Your code is not ready to commit.
- You want to pull latest changes but have uncommitted work.
- You want to clean your working directory temporarily.

3 Common Git Stash Commands

Commond	Purpose
git stash	Stash current changes (tracked files).
git stash -u	Stash including untracked files.
git stash -a	Stash including untracked and ignored files.
git stash list	List all stashes.
git stash apply	Apply the latest stash (keep it).
git stash pop	Apply the latest stash and drop it from the list.
git stash drop stash@{n}	Delete specific stash.
git stash clear	Delete all stashes.

4 Stash Workflow

Mark un changes	Make some changes in your working directory
git stash	Save changes to stash and clean your directory
Switch branch	Go to another branch to do other work
Do your work	Commit / Push / Pull whatever you need
git stash apply / pop	Bring back your stashed changes in a commit
Continue your work	Your changes are back! Keep coding ☺

5 Example Scenario

You are working on feature/login and make some changes.	# Make some changes (not committed)
Need to switch to main for a critical bug.	git stash
Switch to main and fix the bug.	git checkout main
Commit and push the fix.	git commit -m "Fix bug" git push origin main
Come back to feature/login.	git checkout feature/login
Apply your stashed changes and continue coding.	git stash pop

6 Tips & Best Practices

- Always write meaningful commit messages.
- Stash is temporary, not a replacement for commit.
- Use git stash list to keep track of stashes.
- Use pop if you are sure, apply if you want to keep it.
- Don't forget to clear old stashes.

Remember

Stash is like a "Save and Park" for your changes.

Key Point:

Stash helps you stay flexible without losing your work in progress. Use it smartly, not frequently!

Best Practices

- Pull latest changes before starting work.
- Communicate with your team.
- Resolve conflicts as soon as possible.
- Test after resolving conflicts.
- Understand the code before deciding.

Remember

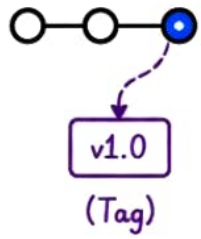
- Conflicts are normal in team work.
- Stay calm and read the changes carefully.
- Understand the code before deciding.
- Clean commits = Happy team! ☺

Git Tag

A Git Tag is used to mark a specific point in history, usually for releases or important milestones.

1 What is Git Tag?

- A tag is like a label for a specific commit.
- Tags are immutable (won't change).
- Used mostly for releases (v1.0, v2.0, etc.).



2 Types of Tags

Lightweight Tag	Simple pointer to a commit.
Annotated Tag	Contains metadata like name, email, date, message.

Tip: Annotated tags are recommended for releases because they contain more information.

3 Common Tag Commands

Commond	Purpose
git tag	List all tags.
git tag <tagname>	Create lightweight tag.
git tag -a <tagname> -m "message"	Create annotated tag.
git show <tagname>	Show tag information.
git push origin <tagname>	Push a specific tag to remote.
git push origin --tags	Push all tags to remote.
git tag -d <tagname>	Delete tag locally.
git push origin --delete <tagname>	Delete tag from remote.

4 Example Workflow

Make changes & commit

`git tag -a v1.0 -m "Release 1.0"` Create annotated tag

`git push origin --tags` Push tag to remote

Check on GitHub — Tag will be visible in releases

5 Example Scenario

Work on new feature
Make changes and commit
`git commit -m "New feature"`

Create a tag
Mark this commit as a release
`git tag -a v1.0 -m "Release 1.0"`

Push the tag
Share the tag with remote
`git push origin --tags`

Tag appears in releases
Tag is visible on GitHub under Releases

6 Tag vs Branch

	Tag	Branch
	Points to a specific commit (fixed)	Points to latest commit (moves)
	Immutable (doesn't change)	Mutable (changes over time)
	Used for releases or milestones	Used for feature development
	Easy to track releases	Used for ongoing work

7 Best Practices

- Use annotated tags for releases.
- Follow a consistent naming convention (v1.0.0, v2.1.0, etc.).
- Push tags to remote to ensure team has access.
- Delete unused tags to keep repo clean.

Version Example
- v1.0.0
- v1.1.0
- v2.0.0-beta
- v2.0.0

Remember

- ✔ Tag = snapshot of history.
- ✔ Use tags for releases.
- ✔ Branches for development.
- ✔ Push tags to share with team.

Tag It Right, Release It Smart!

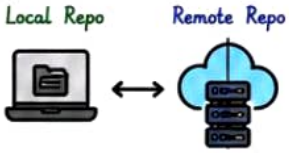
Key Point: Tags help you track important points in your project and make releases easy to manage. Tag your success, release with confidence!

Git Remote

Git Remote is a way to connect your local repository to a remote repository (like GitHub, GitLab, Bitbucket) so you can push, pull and collaborate.

1 What is Git Remote?

- Remote is a version of your project hosted on the internet or network.
- It allows multiple developers to work on the same project.



(GitHub / GitLab / Bitbucket)

2 Common Remotes

GitHub | GitLab | Bitbucket

Tip:
By default, the main remote name is origin.

3 Common Git Remote Commands

Command	Purpose
`git remote`	List all remotes.
`git remote -v`	List all remotes with URLs.
`git remote add <name> <url>`	Add a new remote.
`git remote remove <name>`	Remove a remote.
`git remote rename <old> <new>`	Rename a remote.
`git remote show <name>`	Show details about a remote.

4 Example: Add Remote

- Add remote named 'origin'
  `git remote add origin https://github.com/user/repo.git`
- Verify
  `git remote -v`
- Fetch
  `git fetch https://github.com/user/repo.git (fetch)`
- Push
  `git push origin main https://github.com/user/repo.git (push)`

Remember

origin is just a name.
You can name it anything you want!

origin = Default Remote

5 Push & Pull with Remote

Push Code to Remote

`git push origin <branch>`

Example:

```
git push origin main
```

Pull Code from Remote

`git pull origin <branch>`

Example:

```
git pull origin main
```

6 View Remote Details

```
git remote show origin
remotes origin
Fetch URL:  https://github.com/user/repo.git
Push URL:  https://github.com/user/repo.git
HEAD branch:  main
Remote branches:
  main    tracked
  dev     tracked
```

7 Typical Workflow with Remote

1. Code Locally → 2. Commit Changes → 3. Push to Remote → 4. Team Pulls & Collaborates → 5. Repeat & Sync

8 Best Practices

- ✔ Always pull before you push.
- ✔ Keep your main branch protected.
- ✔ Use meaningful remote names.
- ✔ Regularly fetch & pull to stay updated.
- ✔ Remove unused remotes.

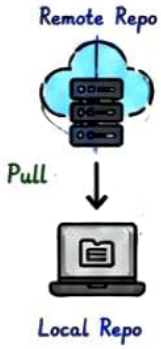
Key Point: Remote connects your local repo to the world.
Use it to collaborate, participate and ship better code together!

Git Pull

Git Pull is used to fetch and download changes from a remote repository and merge them into your current branch.

1 What is Git Pull?

- It gets the latest changes from the remote repository.
- It follows the fetch + merge automatically in one step.
- Keeps your local branch up to date.



2 Git Pull Command

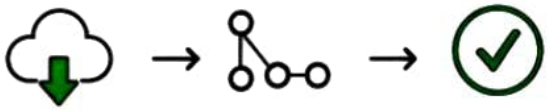
```
git pull <remote> <branch>
```

Example:

```
git pull origin main
```

Tip: If you are on a branch and run "git pull" without specifying a branch, Git will pull from the tracking branch automatically.

3 What Happens During Git Pull?

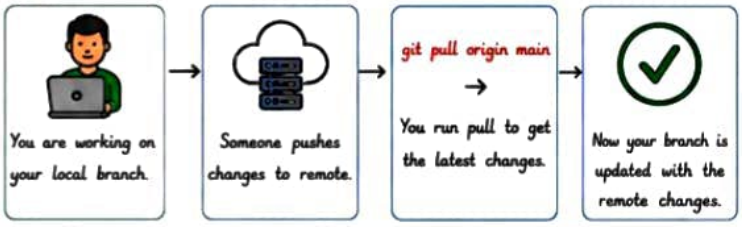


1. **Fetch** — Downloads changes from remote repository.
2. **Merge** — Merges those changes into your current branch.
3. **Update** — Your local branch is now up to date with remote.

4 Pull vs Fetch

Feature	git pull	git fetch
Downloads changes	✓	✓
Merge changes	✓	✗
Updates working directory	✓	✗
Affects local files	✓	✗
Use case	To work & update your code	To check for updates (safe sync)

5 Example Scenario



6 Common Notifications

- Already up to date. : No new updates found.
- Fast-forward : Pulled new changes smoothly.
- Merge made by the 'recursive' strategy. : Git merged the changes.
- Conflicts: : You have to resolve manually.

7 Pull with Rebase (Cleaner History)

Instead of merge commit, you can pull with rebase to keep history clean.

```
git pull --rebase origin main
or
git pull -r origin main
```

8 Best Practices

- ✓ Always pull before you push.
- ✓ Keep your main branch protected.
- ✓ Pull regularly to avoid conflicts.
- ✓ Use meaningful commit messages.
- ✓ Backup your work before pulling important changes.

Remember

- Pull = Fetch + Merge
- Keeps your local branch up to date.
- Helps in smooth collaboration.

Key Point:

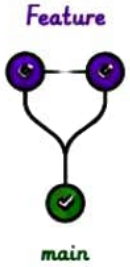
A pull is like getting the latest updates from your team so you all are on the same page!

Git Merge

Git Merge is used to combine changes from one branch into another by integrating the history of both branches.

1 What is Git Merge?

- Merge combines changes from one branch into another.
- It preserves the history.
- Usually, we merge feature branches into main.



2 Merge vs Rebase

Feature	Merge	Rebase
Preserves history	✓	✗
Creates merge commit	✓	✗
Keeps true history	✓	✗
Clean linear history	✗	✓
Safe for shared branches	✓	✗
Good for private branches	✗	✓

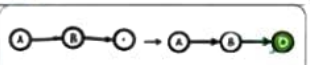
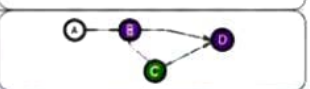
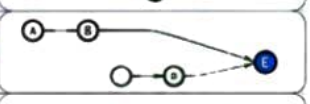
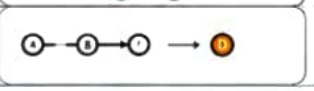
3 Common Merge Commands

Command	Purpose
`git merge <branch>`	Merge <branch> into current branch
`git merge --no-ff <branch>`	Create a merge commit even if fast-forward is possible
`git merge --ff-only <branch>`	Merge only if fast-forward is possible
`git merge --abort`	Abort a merge in progress
`git merge --continue`	Continue merge after resolving conflicts
`git log --graph --oneline`	Show merge history in graph

4 Example Merge Workflow

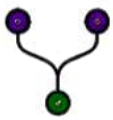
Switch to main or target branch	`git checkout main / develop`
Merge feature branch	`git merge feature/login`
Resolve conflicts (if any)	Edit files and save
Add resolved changes	`git add .`
Commit merge	`git commit`
Push to remote	`git push origin main`

5 Merge Types

Fast-forward		No new commit, just moves pointer.
3-Way Merge		New merge commit created.
Merge Commit		Creates a new merge commit.
Squash Merge		Combines all changes into one commit.

6 Example Scenarios

Merge feature into main



Bring feature changes to main.

Keep history

Use merge when history is important.

Team collaboration

Best for team projects.

Update from remote

Pull latest & merge locally.

7 Resolve Merge Conflicts

- Git will mark conflicted files.
- Open the files and look for conflict markers.
- Edit and keep the correct changes.
- Stage the resolved files (`git add <file>`)
- Commit the merge (`git commit`)

Conflict Markers

```
<<<<<< HEAD
Your changes
=======
Incoming changes
>>>>>> branch-name
```

Remove markers after resolving.

8 Best Practices

- Always pull before you merge.
- Communicate with your team.
- Resolve conflicts as soon as possible.
- Test after merging.
- Use meaningful commit messages.
- Delete merged branches (`git branch -d branch-name`).

Remember

- Merge = Combine histories.
- Merge commit shows the point where branches came together.

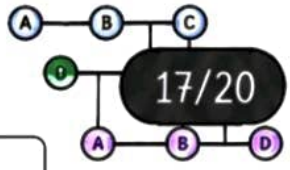
Good history = Easy to understand + Easy to maintain

Key Point:

Merge helps you keep the full history and work together smoothly!

@the.devops111

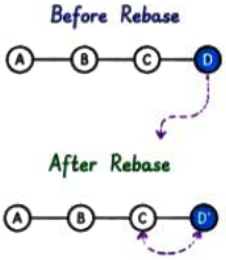
Git Rebase



Git Rebase is used to move or combine a series of commits to a new base commit. It keeps the project history clean and easy to read.

1 What is Git Rebase?

- Rebase moves your branch to a new base.
- It replays your commits on top of the new base.
- It results in a linear and clean history.



2 Rebase vs Merge

Feature	Rebase	Merge
History	Linear & clean ✓	Multiple merge commits ✗
Preserves History	No (rewrites) ✗	Yes ✓
Result	Clean history ✓	Branch structure ✗
Best For	Before sharing ✓	After sharing ✓
Complexity	Medium ⚠	Simple ✓

Tip: Use rebase for feature branches before merging into the main branch.

3 Common Rebase Commands

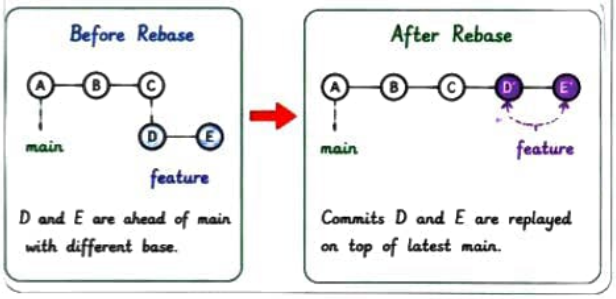
Command	Purpose
`git rebase <base>`	Rebase current branch onto <base>.
`git rebase --continue`	Continue rebase after resolving conflicts.
`git rebase --abort`	Abort rebase and return to original state.
`git rebase --skip`	Skip the current commit and continue.
`git rebase -i <base>`	Interactive rebase.
`git rebase --onto <new> <old> <branch>`	Rebase <branch> onto <new> instead of <old>.

4 Example: Rebase Workflow

- Checkout your feature branch → `git checkout feature/login`
- Rebase onto latest main → `git rebase main`
- Resolve conflicts if any → Edit files → `git add .` `git rebase --continue`
- Push updated branch (force push) → `git push --force-with-lease origin feature/login`

Tip: Always use `--force-with-lease` instead of `--force`.

5 Rebase Example



6 Interactive Rebase

`git rebase -i HEAD~3`

You can use below commands:

- pick : use commit
- reword : edit message
- edit : stop to modify
- squash : merge with previous
- drop : remove commit

6b When to Use Rebase?

- ✓ Cleaning up your commit history before merging.
- ✓ Squashing multiple commits into one.
- ✓ Fixing messy or random commit history.
- ✗ Avoid rebase on shared public branches.

7 Resolve Rebase Conflicts

- ✓ Rebase will stop at conflict.
- ✓ Open the conflicted file.
- ✓ Resolve the conflicts.
- ✓ Stage the resolved file (`git add <file>`).
- ✓ Continue the rebase (`git rebase --continue`).

Example Conflict

```
<<<<<< HEAD (main)
Your changes
======
Incoming changes
>>>>>> feature/login
```

Remove markers after resolving.

8 Best Practices

- ✓ Always pull before you rebase.
- ✓ Use rebase on feature branches, not on main.
- ✓ Communicate with your team.
- ✓ Don't rebase commits that are already pushed to public branches.
- ✓ Use `--force-with-lease` while pushing.
- ✓ Keep your main branch updated regularly.

Remember

- Rebase = Move commits.
- Rebase rewrites history.
- Rebase makes history clean and linear.

Key Point:

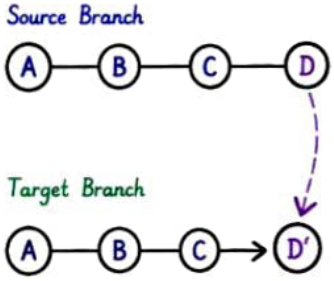
Rebase makes your Git history clean, easy to read and maintain.

Git Cherry-pick

Git Cherry-pick is used to apply a specific commit from one branch to another.

1 What is Cherry-pick?

- Apply a specific commit from one branch to another.
- Creates a new commit with the same changes.
- Keeps history clean and focused.



Commit D from source branch is cherry-picked to target branch as D'.

2 When to Use Cherry-pick?


Pick a bug fix from another branch.


Apply a feature commit without merging the whole branch.

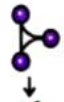

Share only specific changes with team.


Keep production branch stable and pick only required fixes.

3 Common Cherry-pick Commands

Command	Purpose
`git cherry-pick <commit_id>`	Apply the specific commit to current branch.
`git cherry-pick <id1> <id2>`	Apply multiple commits.
`git cherry-pick --no-commit`	Apply changes without committing.
`git cherry-pick --continue`	Continue after resolving conflicts.
`git cherry-pick --abort`	Abort cherry-pick and go back.

4 Cherry-pick Workflow


Checkout target branch — `git checkout main`


Pick commit from source branch — `git cherry-pick <commit_id>`


Resolve conflicts (if any) — Fix files & stage changes


Continue cherry-pick — `git cherry-pick --continue`


Push changes (if needed) — `git push origin main`

Remember

- Cherry-pick only necessary commits.
- Always test after cherry-pick.
- Use on local or private branches, not on public/shared branches.

6 Tip

Use cherry-pick before raising a Pull Request to keep changes small and review easy.

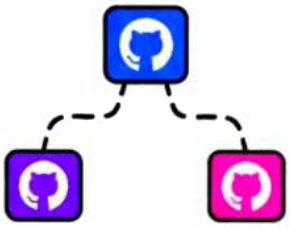
GitHub Basics

GitHub is a platform for collaboration and automation.

1 Repository

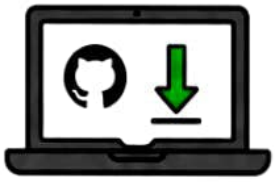
- Store your project files and history.
- A repo can be public or private.

2 Fork



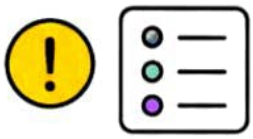
- Copy someone else's repo to your account.
- Useful to contribute to open source projects.

3 Clone



- Download a repo to your machine.
- Work on it locally.

4 Issues



- Track bugs, tasks, ideas or requests.
- Helps in project management.

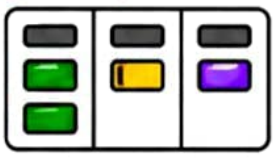
5 Pull Request

- Propose changes and request a review.
- Discuss, review and merge changes.

6 Actions

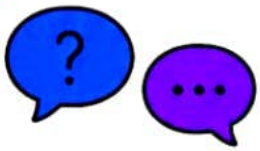
- Automate build, test and deploy (CI/CD).

7 Projects



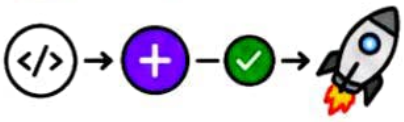
- Organize work and track progress.

8 Discussions



- Discuss ideas and share knowledge.

9 Workflow



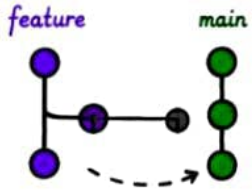
- Code → PR → Review → Merge → Deploy

Tip: Collaborate, review and automate with GitHub to deliver software together!

Pull Requests

Pull Requests let you propose changes, review code, and merge safely.

1 Create PR



Open a Pull Request from your branch to main.

2 Review

Review the changes, check code quality and logic.

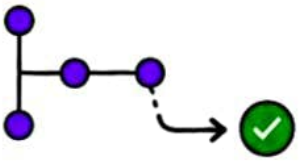
3 Comments

Ask questions, request improvements, and discuss.

4 Approval

Approve the PR when everything looks good.

5 Merge



Merge the PR into main and complete the changes.

6 Close

Close PR if not needed or no longer relevant.

7 Best Practices

- Keep PRs small and focused.
- Write clear title and description.
- Link related issues.
- Review before merging.
- Follow team code standards.
- Test your changes.

PR Checklist

- [x] PR title is clear and meaningful
- [x] Description explains what and why
- [x] Linked to relevant issue (if any)
- [x] Self-review done
- [x] Tests added/updated (if needed)
- [x] No unnecessary commits
- [x] Code follows team standards
- [x] Ready for review

Good PRs make great teams!

Tip

A good Pull Request leads to better code, happier teams, and faster delivery!

Git Best Practices

Good Git habits = Clean code, Happy team, and Successful projects!

1 Small Commits

Write small, focused commits.
One change = one commit.

```
git commit -m "Add login validation"
```

2 Meaningful Messages

Write clear, concise and meaningful commit messages.

- Fix login bug ✓
- Update 123 ✗

3 Branch Naming

- feature/login
- bug/fix-payment
- hotfix/issue-123

Use clear and consistent branch names.

4 Pull Frequently

Always pull latest changes from your branch or main.

```
git pull origin main
```

5 Avoid Force Push

Don't use force push on shared branches. It can overwrite others' work!

```
git push --force ✗
```

6 Review Code

Review code carefully and provide helpful feedback.

Work as a team leader, not a coder.

7 Protect Main

Protect main branch from PR. Use rules to require reviews and pass all checks.

Enable branch protection

8 Checklist

- [x] Code is tested locally
- [x] Meaningful commit message
- [x] PR is properly filled
- [x] Linked issue
- [x] Code follows team standards
- [x] Self-review done
- [x] All checks are green
- [x] Then open PR

9 Good Habits

Good habits bring great code and great teams!

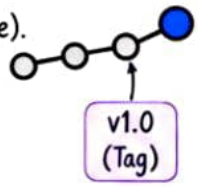
Remember:
Follow good Git practices and become a better developer every day!

Git Tag

A Git Tag is used to mark a specific point in history, usually for releases or important milestones.

1 What is Git Tag?

- A tag is like a label for a specific commit.
- Tags are immutable (can't change).
- Used mainly for releases (v1.0, v2.0, etc.).



2 Types of Tags

Lightweight Tag	Simple pointer to a commit.
Annotated Tag	Contains metadata like name, email, date, message.

Tip: Annotated tags are recommended for releases because they contain more information.

3 Common Tag Commands

Commond	Purpose
git tag	List all tags
git tag <name>	Create a lightweight tag
git tag -a <name> -m "msg"	Create an annotated tag
git tag -f <name>	Force move tag
git tag -d <name>	Delete a tag
git push --tags	Push all tags to remote
git fetch --tags	Fetch all tags from remote

4 Example Workflow

Make changes & commit

↓

Create a tag (e.g., v1.0)

↓

Push the tag to remote

Tip: Use tags for stable releases and production deployments.

5 Example Scenario

- You finish a new feature.
- Create a tag v1.2 for this release.
- Push the tag to GitHub.
- Now anyone can check out or deploy exactly v1.2.

6 Best Practices

- Use meaningful tag names (v1.0, v1.1).
- Prefer annotated tags for releases.
- Don't delete tags (instead create a new one).
- Keep tags consistent across teams.

Key Point:
Tags are immutable labels that mark important points in your Git history.

Remember:
Use tags for releases, not for everyday development.

Tag it. Release it. Done!

Github @Nraajsingh55-dot

GIT COMMANDS CHEAT SHEET

Most Important Git Commands for DevOps & Developers

NO.	COMMAND	WHAT IS IT?	WHY DO WE USE IT?	EXAMPLE
1.	`git init`	Initializes a new Git repository	Start version control in your project	`git init`
2.	`git clone`	Clone a remote repository to your local machine	To work on an existing project	`git clone <repo-url>`
3.	`git status`	Show the working tree status	To see modified, staged and untracked files	`git status`
4.	`git add`	Stage changes for next commit	Add modified files to be committed	`git add <file>` `git add .`
5.	`git commit`	Save staged changes with a commit message	Save your work with a message	`git commit -m "message"`
6.	`git log`	Show commit history	To review project history	`git log`
7.	`git diff`	Show changes between working dir, staging & commits	To review changes before committing or merging	`git diff` `git diff --staged`
8.	`git branch`	List, create or delete branches	To manage branches	`git branch` `git branch <name>`
9.	`git checkout`	Switch branches or restore files (older command)	To switch context or restore old versions	`git checkout <branch>` `git checkout -- <file>`
10.	`git merge`	Merge branches into current one	To integrate feature branches	`git merge <branch>`
11.	`git rebase`	Reapply commits on top of another branch	To keep history clean and linear	`git rebase <branch>`
12.	`git stash`	Save changes in a stash (temporarily)	To save WIP changes (temporarily) and switch	`git stash` `git stash pop | apply`
13.	`git pull`	Fetch and integrate changes from a remote repository	To update local repo with remote changes	`git pull <remote> <branch>`
14.	`git push`	Push local commits to remote repository	To share your work with others	`git push <remote> <branch>`
15.	`git remote`	Manage remote repositories	To add, view or remove remote repos	`git remote -v`
16.	`git fetch`	Fetch changes from remote repository	To download new changes without merging	`git fetch <remote>`
17.	`git revert`	Revert a commit by creating a new commit	To safely undo changes without rewriting history	`git revert <commit-id>`
18.	`git cherry-pick`	Apply a specific commit from one branch to another	To take one commit without merging all	`git cherry-pick <commit-id>`
19.	`git switch`	Switch to an existing branch (modern command)	To switch between branches	`git switch <branch>`
20.	`git switch -c`	Create a new branch and switch to it	To create and move to a new branch	`git switch -c <new-branch>`
21.	`git tag`	Create a tag for a specific commit	To mark releases or important versions	`git tag <tag-name>`
22.	`git push --tags`	Push tags to remote	To share tags with others	`git push --tags`
23.	`git show`	Show details of a commit	To see what a commit contains	`git show <commit-id>`

24.	`git reset`	Reset current HEAD to a specific state	To undo changes or revert commits	`git reset <file>`
				`git reset --soft|--hard <commit>`
				⚠ `--hard` can discard changes.

25.	`git clean`	Remove untracked files and directories	To clean up untracked files	`git clean -n (dry run)` `git clean -f (force clean)`

RESET TYPES EXPLAINED

`git reset --soft <commit>` → A → B → C
(keeps changes staged)

`git reset --mixed <commit>` → A → B → C
(keeps changes unstaged)

`git reset --hard <commit>` → A → B → C
(discard all changes)

⚠ Use `--hard` carefully. It can DELETE uncommitted changes permanently.

REVERT vs RESET

REVERT

- Creates a new commit
- Safe, doesn't change history
- Use for public branches

A → B → C → D → E

RESET

- Moves HEAD to previous commit
- Changes history
- Use for local branches

A → B → C → D → L

COMMON WORKFLOW

1. `git clone <repo>`
2. `git checkout -b feature`
3. `git add .`
4. `git commit -m "message"`
5. `git pull origin feature`
6. `git push origin feature`
7. Create Pull Request

GOOD PRACTICES

- ✓ Commit small changes often
- ✓ Write clear commit messages
- ✓ Use feature branches
- ✓ Pull before push
- ✓ Keep your repo clean always

> "Git is not just a tool, it's a time machine for your code."

Happy Coding!

TIPS

- Use `git status` to stay updated
- Use `git log` to understand history
- Use `git stash` for quick switch
- Use `git reset` for local undo
- Use `git diff` to review before commit